



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Практическая работа № 6/4 ч.

Разработка мобильных приложений

(наименование дисциплины (модуля) в соответствии с учебным планом)

Уровень

бакалавриат

(бакалавриат, магистратура, специалитет)

Форма обучения

очная

(очная, очно-заочная, заочная)

Направление(-я)

подготовки

09.03.02 «Информационные системы и технологии»

(код(-ы) и наименование(-я))

Институт

кибербезопасности и цифровых технологий

(полное и краткое наименование)

Кафедра

КБ-14 «Цифровые технологии обработки данных»

(полное и краткое наименование кафедры, реализующей дисциплину (модуль))

Используются в данной редакции с учебного года

2024/25

(учебный год цифрами)

Проверено и согласовано «___» _____ 20__ г.

(подпись директора Института/Филиала
с расшифровкой)

Москва 2024 г.

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ.....	4
1. ЗНАКОМСТВО С FRAGMENT.....	8
2. FRAGMENT MANAGER.....	12
3. ПЕРЕДАЧА ДАННЫХ.....	16
4. КОНТРОЛЬНОЕ ЗАДАНИЕ.....	22

ВВЕДЕНИЕ

Одной из особенностей создания приложений для Android является то, что одно и то же приложение может запускаться на устройствах с разными экранами и процессорами и будет работать на них одинаково. Но это вовсе не означает, что оно будет на них одинаково выглядеть.

Чтобы интерфейсы приложения на телефоне и планшете отличались друг от друга, требуется определить разные макеты для больших и малых устройств. Однако определить разные макеты для разных устройств недостаточно. Чтобы приложение работало по-разному в зависимости от устройства, наряду с разными макетами должен выполняться разный код Java. Например, в приложении на рисунке 1 необходимо предоставить одну активность для планшетов и две активности для телефонов.

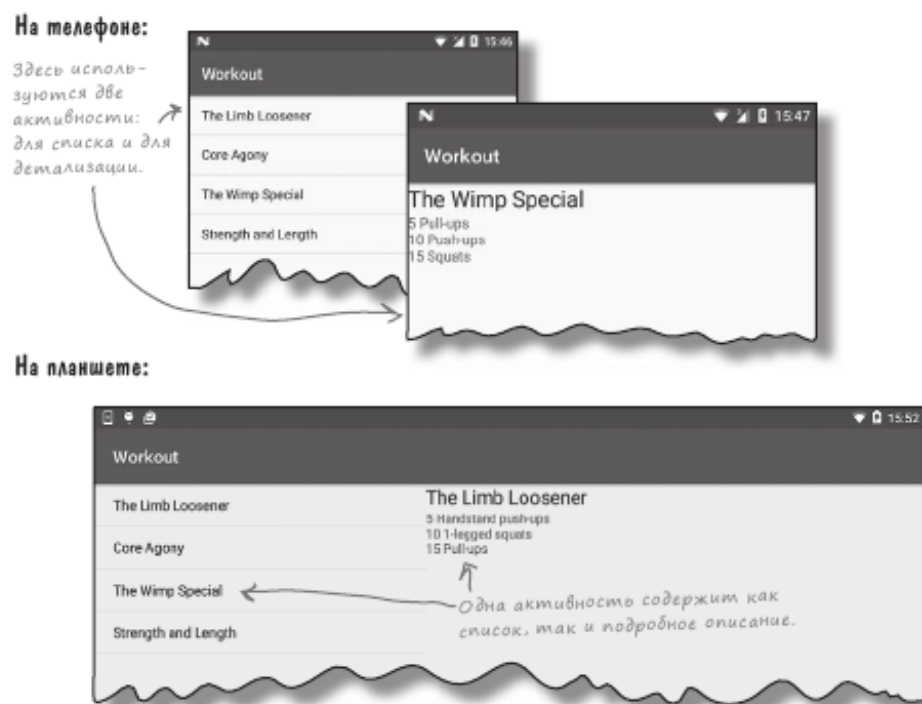
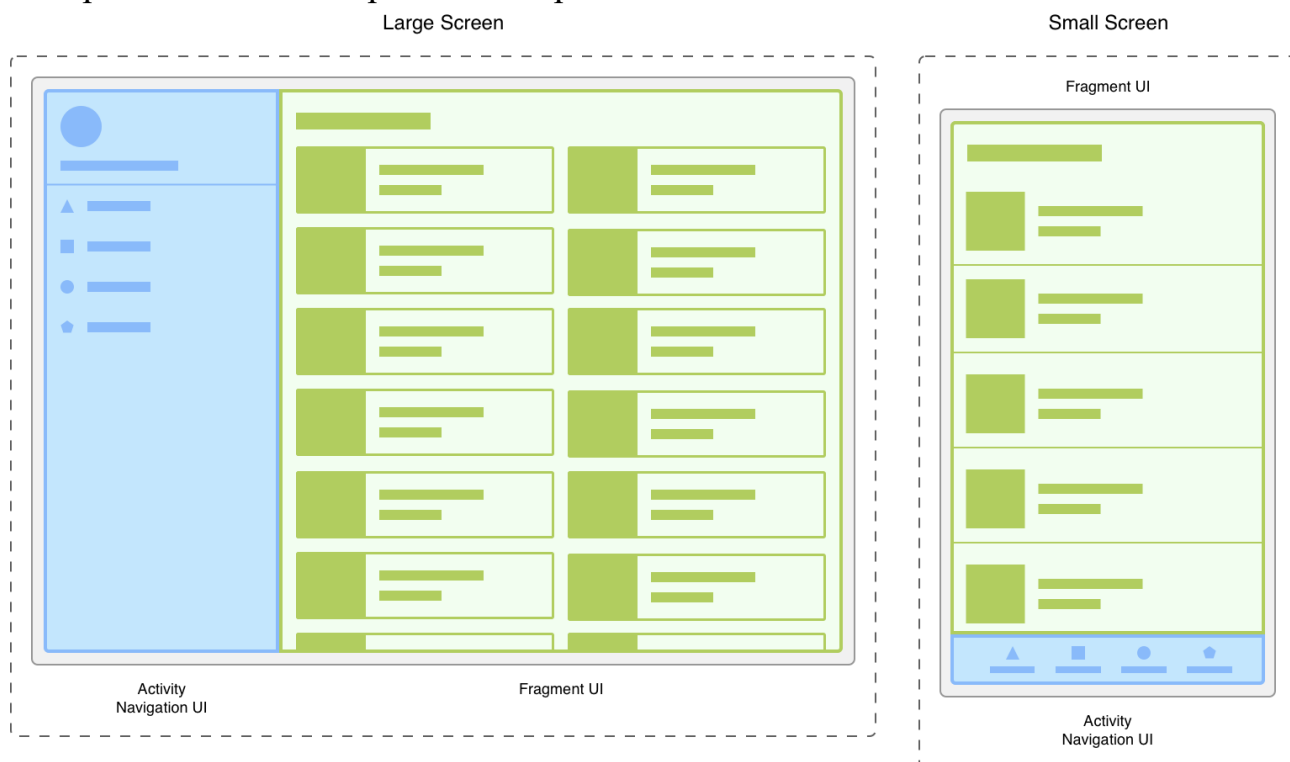


Рисунок 1 – Внешний вид приложения на разных устройствах

Данный способ может привести к дублированию кода второй активности, которая работает только на телефонах. Один код должен выполняться в нескольких активностях. Вместо того, чтобы дублировать код в двух активностях, следует использовать фрагменты (fragments). Что же собой представляет фрагмент?

Фрагмент (Fragment) является важным компонентом Android-приложений и представляет собой повторно используемую часть пользовательского интерфейса. Он определяет и управляет собственным макетом, имеет собственный жизненный цикл, обрабатывает собственные события. Фрагменты

привносят модульность и возможность повторного использования в рамках одной Activity, позволяя разделить пользовательский интерфейс на отдельные части. Глобальные элементы пользовательского интерфейса (например, навигационная панель) удобно располагать в Activity, а интерфейс отдельного экрана или его части предпочтительно определять в рамках фрагмента. В качестве примера возможно рассмотреть приложение с навигацией и отображением списка сущностей, имеющее различный интерфейс в зависимости от размера экрана. Отделение элементов навигации от содержимого позволяет более удобно управлять состоянием приложения, так как Activity отвечает за отображение правильного пользовательского интерфейса навигации, а фрагмент отображает список с правильным расположением.



Фрагмент может иметь свою разметку, но возможно использование и без неё. Также у фрагмента есть свой жизненный цикл, во многом совпадающий с жизненным циклом активности. Пожалуй, это единственное сходство с активностью.

Фрагменты в Android были введены для улучшения гибкости и удобства разработки пользовательского интерфейса, особенно для мобильных устройств с различными размерами экранов и конфигурациями. Основные причины появления фрагментов, следующие:

1. Управление многократными компонентами UI

Фрагменты позволяют разделить сложные интерфейсы на более мелкие и переиспользуемые компоненты. Это помогает упрощать логику отображения различных элементов на экране, особенно если приложение должно

адаптироваться под различные размеры экранов (например, телефоны и планшеты).

2. Гибкость в построении интерфейса

С фрагментами легче адаптировать интерфейс под разные типы устройств. Например, на планшете можно разместить несколько фрагментов в одном экране, а на телефоне отображать только один фрагмент в данный момент времени. Это облегчает создание интерфейсов, которые могут динамически изменяться в зависимости от размеров экрана и ориентации устройства.

3. Модульность и повторное использование

Фрагменты позволяют разделить приложение на небольшие модули, которые можно переиспользовать в разных частях приложения. Это упрощает как разработку, так и тестирование. Например, фрагмент, который отображает список, может быть использован в разных активностях или экранах.

4. Управление состоянием и жизненным циклом

Фрагменты обладают собственным жизненным циклом, который можно контролировать независимо от жизненного цикла активности, в которой они находятся. Это позволяет более точно управлять состоянием интерфейса, например, сохранять и восстанавливать состояние при смене ориентации экрана или других событиях, которые могут повлиять на работу приложения.

5. Снижение сложности работы с многозадачностью

Когда приложение использует несколько фрагментов, оно может динамически менять свой интерфейс в зависимости от того, какой фрагмент должен быть активен в данный момент. Это упрощает работу с многозадачностью и позволяет плавно переключаться между разными частями интерфейса без необходимости перезапускать активности.

6. Переходы и анимации

Фрагменты поддерживают анимации переходов между различными состояниями или экранами. Это позволяет создавать более плавный и приятный пользовательский опыт, а также улучшает взаимодействие с пользователем. Фрагменты позволяют значительно улучшить архитектуру и пользовательский интерфейс приложения, обеспечивая удобство разработки, гибкость в адаптации под различные устройства и лучшую работу с состоянием и многозадачностью.

Так же как активити, фрагменты должны реализовывать некоторые методы обратного вызова жизненного цикла, которые позволяют управлять его состоянием, таким как его добавление или удаление из активити, и для обработки переходов между состояниями жизненного цикла активити. Например, когда для активити вызывается *onPause()* метод, все фрагменты данной активити также получают вызов *onPause()*. Жизненный цикл компонента *Fragment* представлен на рисунке 3 вместе с циклом *Activity*.

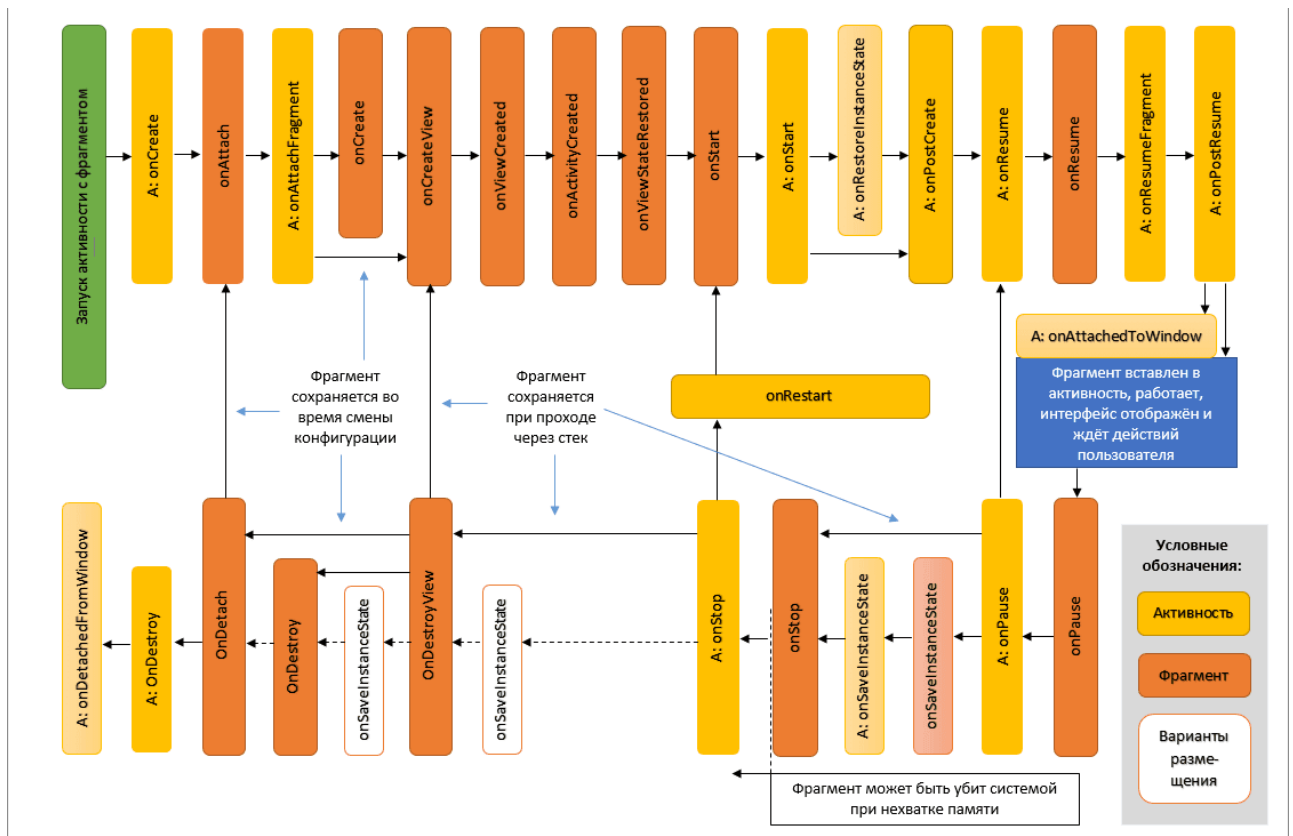


Рисунок 2 – Жизненный цикл Fragment совместно с Activity

Как правило, требуется реализовать следующие методы жизненного цикла:

- `onCreate()` – система вызывает данный метод, когда создает фрагмент.

В своей реализации разработчик должен инициализировать ключевые компоненты фрагмента, которые требуется сохранить, когда фрагмент находится в состоянии паузы или возобновлен после остановки.

- `onCreateView()` – система вызывает данный метод при первом отображении пользовательского интерфейса фрагмента на дисплее. Для прорисовки пользовательского интерфейса фрагмента следует вернуть из этого метода объект `View`, который является корневым в макете фрагмента. Если фрагмент не имеет пользовательского интерфейса, можно вернуть `null`.

- `onPause()` – система вызывает данный метод как первое указание того, что пользователь покидает фрагмент (это не всегда означает уничтожение фрагмента). Обычно именно в этот момент необходимо фиксировать все изменения, которые должны быть сохранены за рамками текущего сеанса работы пользователя (поскольку пользователь может не вернуться назад).

Один и тот же класс фрагмента может быть использован в нескольких экземплярах как в рамках одной `Activity`, так и в разных. При этом важно следить, чтобы каждый фрагмент содержал только ту логику, которая необходима для управления его собственным пользовательским интерфейсом, поэтому не следует допускать явных зависимостей одних фрагментов от других.

1. ЗНАКОМСТВО С FRAGMENT

Задание. Создать новый модуль. В меню «File / New / New Module / Phone & Tablet Module / Empty Views Activity». Название модуля «FragmentApp». Необходимо реализовать приложение, позволяющее просматривать список дел и статус их выполнения.

Для использования фрагментов необходимо добавить зависимость *AndroidX Fragment*. Для ее использования в файл *settings.gradle* проекта должен быть добавлен репозиторий Google Maven.

```
dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
        google()
        mavenCentral()
        jcenter() // Warning: this repository is going to shut down soon
    }
}
```

Чтобы включить библиотеку фрагментов *AndroidX* в проект необходимо добавьте следующие зависимости в файл *build.gradle* вашего приложения в зависимости от используемого языка.

```
dependencies {
    def fragment_version = "1.8.5"

    // Java language implementation
    implementation "androidx.fragment:fragment:$fragment_version"
    // Kotlin
    implementation "androidx.fragment:fragment-ktx:$fragment_version"
}
```

Создание статического фрагмента требует выполнение следующих шагов:

- создание XML макета;
- создание дочернего класса от `androidx.fragment.app.Fragment`;
- указание раздуваемого макета XML в методе `onCreateView()` фрагмента.

Имеются специальные виды фрагментов, предназначенные под определённые задачи - *ListFragment*, *DialogFragment*, *PreferenceFragment* и другие.

Имеется два варианта использования фрагментов в приложении:

- в разметке указывается фрагмент с помощью тега *fragment*;
- динамическое подключение фрагмента. В разметку помещается макет из группы `ViewGroup`, который становится контейнером для фрагмента. Обычно, для данной цели используют `FrameLayout`, но это не обязательное условие. В нужный момент фрагмент замещает контейнер и становится частью разметки.

Чтобы создать фрагмент, требуется наследоваться от класса `AndroidX Fragment` и переопределить его методы для реализации собственной логики по аналогии с классом `Activity`. Для создания фрагмента средствами IDE следует воспользоваться меню «New - Fragment - Fragment (Blank)», это создаст файл разметки и файл класса фрагмента. Библиотека `Fragment` также предоставляет

более специализированные базовые классы фрагментов, например `DialogFragment` для отображения диалоговых окон.

```
public class BlankFragment extends Fragment {

    @Override
    public View onCreateView(LayoutInflater inflater, ViewGroup container,
        Bundle savedInstanceState) {
        Log.d(BlankFragment.class.getSimpleName(), "onCreateView");
        // Inflate the layout for this fragment
        View view = inflater.inflate(R.layout.fragment_blank, container, false);
        return view;
    }
}
```

Как правило, для добавления представления фрагмента в представление *Activity*, активность должна наследоваться от *FragmentActivity*, которая является базовым классом для *AppCompatActivity*. В файл разметки *Activity* необходимо добавить *FragmentContainerView*, который определяет место размещения фрагмента.

```
<androidx.fragment.app.FragmentContainerView
    android:id="@+id/fragment_container_view"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:name="com.mirea.fragment.BlankFragment"/>
```

Атрибут *android:name* указывает имя класса фрагмента. При «раздувании» (от англ. *inflate*) макета *Activity*, у фрагмента вызывается метод *onInflate()* и создается транзакция для добавления фрагмента. Чтобы программно добавить фрагмент, атрибут *android:name* не указывается, вместо этого используется явная транзакция для создания фрагмента и добавления его в *Activity*.

```
public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        if (savedInstanceState == null) {
            getSupportFragmentManager().beginTransaction()
                .setReorderingAllowed(true)
                .add(R.id.fragment_container_view, BlankFragment.class, null)
                .commit();
        }
    }
}
```

В метод *add* передаются значения идентификатора *ViewGroup* контейнера и имя класса фрагмента, после чего возможно применить транзакцию с помощью метода *commit()*. Метод *setReorderingAllowed(true)* оптимизирует изменения состояния фрагментов, участвующих в транзакции, чтобы обратный стек, анимация и переходы работали правильно. Включение *setReorderingAllowed(true)* гарантирует, что если несколько транзакций выполняются одновременно, любые промежуточные фрагменты

(т. е. те, которые добавляются, а затем немедленно заменяются) не претерпевают изменений жизненного цикла или не выполняется их анимации или переход.

Обратите внимание, что в предыдущем примере транзакция фрагмента создается только в том случае, если *savedInstanceState* имеет значение *null*. Это необходимо для того, чтобы фрагмент добавлялся только один раз при первом создании активности. Когда происходит изменение конфигурации и пересоздание активности, *savedInstanceState* не равно *null*, и фрагмент не нужно добавлять второй раз, поскольку фрагмент автоматически восстанавливается из *savedInstanceState*.

Одним из способов передачи фрагменту данных является использование объекта *Bundle*.

Задание. Передайте во фрагмент свой номер по списку. В случае получения ошибки при запуске приложения, убедитесь, что в файле разметки *main_activity* не указано поле *name*.

```
public class MainActivity extends AppCompatActivity{

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        if (savedInstanceState == null) {
            Bundle bundle = new Bundle();
            bundle.putInt("my_number_student", 0);

            getSupportFragmentManager().beginTransaction()
                .setReorderingAllowed(true)
                .add(R.id.fragment_container_view, BlankFragment.class, null)
                .commit();
        }
    }
}
```

Получить данные можно с помощью вызова *requireArguments()*.

```
public class BlankFragment extends Fragment {
    ...
    @Override
    public void onViewCreated(@NonNull View view, Bundle savedInstanceState) {
        int numberStudent = requireArguments().getInt("my_number_student");
        Log.d(BlankFragment.class.getSimpleName(), String.valueOf(numberStudent));
    }
    ...
}
```

Фрагменты возможно размещать в активности с указанием поля *name* в файле разметки.

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.appcompat.widget.LinearLayoutCompat
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    tools:context=".MainActivity">

    <fragment
        android:id="@+id/headerFragment"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="1"
        android:name="com.mirea.fragment.HeaderFragment"/>

    <fragment
        android:id="@+id/detailsFragment"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        android:layout_weight="2"
        android:name="com.mirea.fragment.BlankFragment"/>

</androidx.appcompat.widget.LinearLayoutCompat>
```

2. FRAGMENT MANAGER

Задание. Создать новый модуль. В меню «File / New / New Module / Phone & Tablet Module / Empty Views Activity». Название модуля «FragmentManagerApp».

FragmentManager является ключевым компонентом при построении навигации в приложении. Он отвечает за выполнение действий над фрагментами приложения, таких как их добавление, удаление или замена, а также добавление их в стек. Все операции, выполняемые *FragmentManager*, происходят внутри «транзакции», как в операции с базой данных.

Получить доступ к *FragmentManager* возможно из *Activity* или из фрагмента. *FragmentManager* и его подклассы, имеют доступ к *FragmentManager* через метод *getSupportFragmentManager()*. Фрагменты могут содержать один или несколько дочерних фрагментов, внутри фрагмента можно получить ссылку на *FragmentManager*, управляющий дочерними фрагментами, с помощью метода *getChildFragmentManager()*. Для получения доступа к главному *FragmentManager* требуется использовать *getParentFragmentManager()*.

На рисунке показаны два примера, каждый из которых имеет одну *Host Activity*. Она отображает навигацию, которая отвечает за смену фрагмента на разных экранах приложения. Каждый экран реализуется как отдельный фрагмент. На рисунке 4 представлены отношения между фрагментами, их хостами и экземплярами *FragmentManager*.

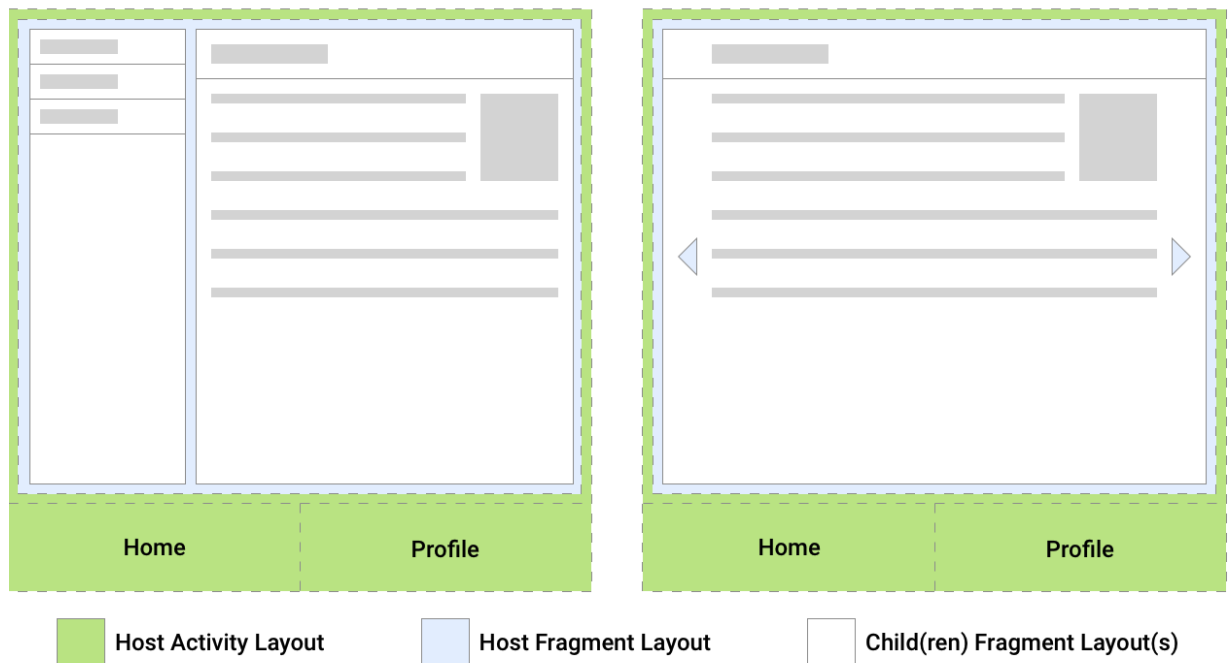


Рисунок 4 – Два примера макета пользовательского интерфейса, показывающие взаимосвязи между фрагментами и действиями их хоста

Activity в обоих этих примерах отображает пользователю навигацию верхнего уровня в виде *BottomNavigationView*, который отвечает за замену

фрагмента хоста различными экранами в приложении. Каждый экран реализован как отдельный фрагмент. Левый пример содержит два дочерних фрагмента, которые образуют разделение экрана на две части, правый - один дочерний фрагмент, составляющий фрагмент экрана пролистывания. *Activity* имеет связанный с ней *FragmentManager*, который управляет дочерними фрагментами. Далее показаны взаимосвязи между *supportFragmentManager*, *parentFragmentManager* и *childFragmentManager*.

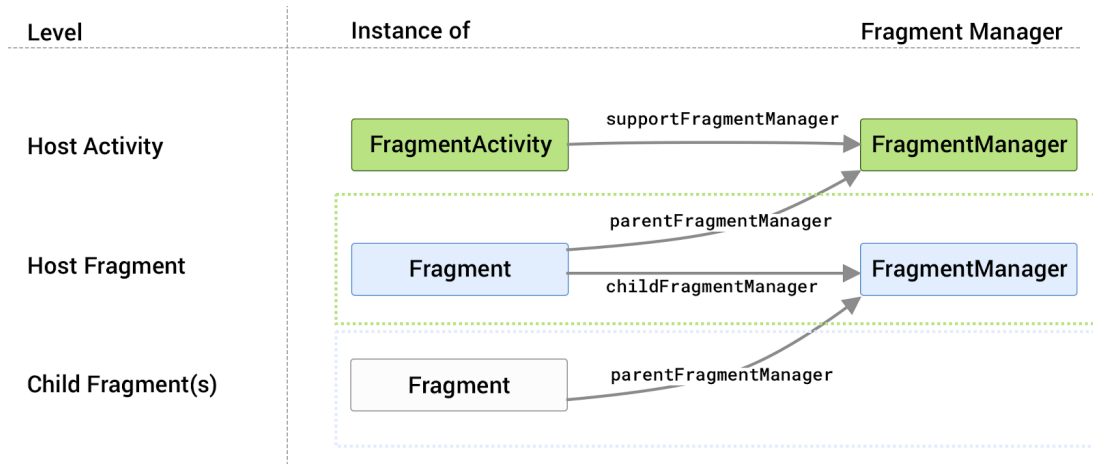


Рисунок 5 – С каждым хостом связан собственный *FragmentManager*, который управляет дочерними фрагментами.

FragmentManager управляет бэк-стеком (механизм управления историей навигации между экранами). Во время выполнения *FragmentManager* может добавлять или удалять фрагменты, каждый набор изменений фиксируется как единая транзакция. Когда пользователь нажимает кнопку Back на устройстве или когда вызывается *popBackStack()*, самая верхняя транзакция фрагмента выгружается из стека. Если в стеке больше нет транзакций фрагментов, и не используются дочерние фрагменты, событие Back переходит в *Activity*.

Все операции, выполняемая *FragmentManager*, происходит внутри «транзакции», как в операции с базой данных. При работе с транзакциями возможно сгруппировать несколько действий в одну транзакцию. Сохранение транзакции в стеке позволяет пользователю перемещаться назад по изменениям фрагментов.

Чтобы получить экземпляр *FragmentTransaction* из *FragmentManager*, требуется вызвать метод *beginTransaction()*. Последний вызов каждой транзакции должен зафиксировать ее с помощью метода *commit()*, который сигнализирует о том, что все операции были добавлены в транзакцию.

```

getSupportFragmentManager().beginTransaction()
    .setReorderingAllowed(true)
...
    .commit();
  
```

Стоит заметить, что вызов *commit()* не приводит к немедленному выполнению транзакции, он планирует ее к выполнению в главном потоке.

Чтобы немедленно запустить транзакцию фрагмента, можно вызвать *commitNow()*, однако его следует использовать только в исключительных случаях. К основным методам класса *FragmentManager*, относятся *add()*, *remove()* и *replace()*.

Метод *add()* позволяет добавить фрагмент, на вход он получает значение идентификатора *ViewGroup* контейнера и класс фрагмента, добавленный фрагмент переводится в состояние RESUMED.

```
getSupportFragmentManager().beginTransaction()  
    .add(R.id.headerContainerView, headerFragment, "header")  
    .commit();
```

Метод *remove()* позволяет удалить фрагмент. Получить его экземпляр можно с помощью методов *findFragmentById()* или *findFragmentByTag()*. Если представление фрагмента было ранее добавлено в контейнер, то в этот момент представление удаляется из контейнера, удаленный фрагмент переходит в состояние DESTROYED.

Метод *replace()* позволяет заменить существующий фрагмент в контейнере новым. Вызов *replace()* эквивалентен вызову *remove()* и *add()*.

По умолчанию изменения, сделанные в транзакции, не добавляются в бэк-стек. Чтобы сохранить эти изменения, требуется использовать метод *addToBackStack()*, который принимает необязательное имя для состояния бэк-стека. Пользователь может отменить транзакцию и вернуть предыдущий фрагмент, нажав кнопку «Назад». Если было добавлено или удалено несколько фрагментов в рамках одной транзакции, все эти операции будут отменены при извлечении обратного стека. Необязательное имя, указанное в вызове *addToBackStack()* дает возможность вернуться к определенной транзакции с помощью *popBackStack()*.

Если не осуществлять обработку *addToBackStack()* при выполнении транзакции, то удаленный фрагмент уничтожается при фиксации транзакции, и пользователь не может вернуться к нему. Если производится вызов *addToBackStack* при удалении фрагмента, то фрагмент переходит в состояние STOPPED, а затем RESUMED, когда пользователь возвращается назад.

```
getSupportFragmentManager().beginTransaction()  
    .replace(R.id.headerContainerView, HeaderFragment.class, null)  
    .setReorderingAllowed(true)  
    .addToBackStack("name") // Name can be null  
    .commit();
```

В этом примере *HeaderFragment* заменяет фрагмент (если таковой имеется) который в данный момент находится в контейнере макета, указанном идентификатором *R.id.headerContainerView*. Предоставление класса фрагмента методу *replace()* позволяет *FragmentManager* обрабатывать экземпляры с помощью *FragmentManager*.

Чтобы показать или скрыть фрагменты, добавленные в контейнер, можно использовать методы `show()` и `hide()`, они устанавливают видимость представлений фрагмента, не влияя на жизненный цикл фрагмента.

Для присоединения и отсоединения фрагментов следует использовать методы `attach()` и `detach()` соответственно. Метод `attach()` повторно присоединяет фрагмент - представление создается заново, прикрепляется к пользовательскому интерфейсу и отображается. Метод `detach()` отсоединяет фрагмент от пользовательского интерфейса, при этом фрагмент остается в том же состоянии, в котором он был помещен в бэк-стек, и находится под управлением *FragmentManager*. Поскольку транзакция рассматривается как единый атомарный набор операций, вызовы `detach` и `attach` для одного и того же экземпляра фрагмента в одной транзакции эффективно отменяют друг друга, что позволяет избежать разрушения и немедленного воссоздания пользовательского интерфейса фрагмента.

Только один *FragmentManager* может управлять обратным стеком фрагмента в любой момент времени. Если приложение отображает на экране несколько одноуровневых фрагментов одновременно или если приложение использует дочерние фрагменты, то один *FragmentManager* предназначен для обработки основной навигации вашего приложения.

Чтобы определить основной фрагмент навигации внутри транзакции фрагмента, необходимо осуществить вызов метода `setPrimaryNavigationFragment()` для транзакции, передав экземпляр управляемого фрагмента `childFragmentManager`.

При добавлении фрагмента имеется возможность создания экземпляра фрагмента вручную и добавить его в `FragmentTransaction`.

```
if (savedInstanceState == null) {  
    HeaderFragment headerFragment = HeaderFragment.newInstance("MIREA");  
    getSupportFragmentManager().beginTransaction()  
        .setReorderingAllowed(true)  
        .add(R.id.headerContainerView, headerFragment, "header")  
        .add(R.id.detailsContainerView, new DetailsFragment(), "details")  
        .commit();  
}
```

Задание. Добавьте в приложение несколько фрагментов. Создайте разметку для них. Первый фрагмент должен содержать какой-то список (например страны). Второй детальную информацию о выбранном пункте. Передача данных будет рассмотрена далее.

3. ПЕРЕДАЧА ДАННЫХ

Для полноценного функционирования фрагментов требуется реализовывать механизм обмена информацией как между *Activity* и ее фрагментами, так и или между двумя или более фрагментами. Чтобы сохранить автономность фрагментов, не позволяйте фрагментам напрямую взаимодействовать с другими фрагментами или с активностью. Библиотека *Fragment* предоставляет два варианта взаимодействия: использование общей *ViewModel* и *Fragment Result API*. Первый предпочтителен для постоянного обмена данными, для получения единожды результата следует использовать *Result API*.

ViewModel

идеальный выбор, когда необходимо совместно использовать данные между несколькими фрагментами или между фрагментами и активностью их хоста. Объекты *ViewModel* хранят данные пользовательского интерфейса и управляют ими. В качестве примера реализуем класс *ShareViewModel*, хранимые данные обернуты в класс *MutableLiveData*, который позволяет изменять их значение.

```
public class ShareViewModel extends ViewModel {  
    private final MutableLiveData<String> selectedItem = new MutableLiveData<>();  
    public void setSomeValue(String item) {  
        selectedItem.setValue(item);  
    }  
    public LiveData<String> getSomeValue() {  
        return selectedItem;  
    }  
}
```

В этом примере данные заключены в класс *MutableLiveData*. Фрагмент, и его хост-активити могут получить общий экземпляр *ViewModel*, передав действие в конструктор *ViewModelProvider*. *ViewModelProvider* обрабатывает создание экземпляра *ViewModel* или его получение, если оно уже существует. Оба компонента могут наблюдать и изменять эти данные.

Рассмотрим вариант взаимодействия между фрагментом и *Activity*. Оба класса могут получить общий экземпляр *ViewModel* с областью видимости *Activity*, передав *Activity* в конструктор *ViewModelProvider*, который обрабатывает создание *ViewModel* или ее получение, если она уже существует.

```

public class MainActivity extends AppCompatActivity implements FragmentListener{
    private ShareViewModel viewModel;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        viewModel = new ViewModelProvider(this).get(ShareViewModel.class);
        viewModel.getSelectedItem().observe(this, item -> {
            // Perform an action with the latest item data.
        });

        HeaderFragment headerFragment = HeaderFragment.newInstance("MIREA");

        if (savedInstanceState == null) {
            getSupportFragmentManager().beginTransaction()
                .setReorderingAllowed(true)
                .add(R.id.headerContainerView, headerFragment, "header")
                .add(R.id.detailsContainerView, new DetailsFragment(), "details")
                .commit();
        }
    }
}

```

По клику на кнопку, расположенную в фрагменте, происходит обновление значения `viewModel` и в Activity возможно получить данное значение.

```

public class HeaderFragment extends Fragment {
    private ShareViewModel viewModel;

    @Override
    public void onViewCreated(@NonNull View view, @Nullable Bundle savedInstanceState) {
        super.onViewCreated(view, savedInstanceState);
        viewModel = new ViewModelProvider(requireActivity()).get(ShareViewModel.class);
        view.findViewById(R.id.buttonSelectItem)
            .setOnClickListener(
                v -> viewModel.selectItem("Some data"));
    }
}

```

Двум или более фрагментам одной и той же деятельности часто необходимо взаимодействовать друг с другом. Например, один фрагмент отображает список, а другой позволяет пользователю применять к списку различные фильтры. Реализация данного случая не является тривиальной без прямого взаимодействия фрагментов, но тогда они перестают быть самодостаточными. Кроме того, оба фрагмента должны обрабатывать сценарий, в котором другой фрагмент еще не создан или не виден. Данные фрагменты могут совместно использовать *ViewModel* используя свою область действия для обработки этого взаимодействия. При таком совместном использовании *ViewModel* фрагментам не нужно знать друг о друге, а активности не нужно ничего делать для облегчения взаимодействия.

Поскольку фрагменты используют одну и ту же область видимости, они получают один и тот же экземпляр *ViewModel*, что позволяет им взаимодействовать друг с другом, при этом фрагменты напрямую не знают друг о друге.

```

public class DetailsFragment extends Fragment {
    private ShareViewModel viewModel;
    @Override
    public void onCreateView(@NonNull View view, @Nullable Bundle savedInstanceState) {
        super.onCreateView(view, savedInstanceState);
        viewModel = new ViewModelProvider(requireActivity()).get(ShareViewModel.class);
        viewModel.getSelectedItems().observe(getViewLifecycleOwner(), data -> {
            Log.d(DetailsFragment.class.getSimpleName(), data);
        });
    }
}

```

Для обеспечения обмена данными между дочерними фрагментами и родительским фрагментом используется аналогичный механизм, но в качестве области видимости `ViewModel` следует использовать родительский фрагмент, а получить к нему доступ можно с помощью метода `requireParentFragment()`.

Задание. На предыдущем шаге были созданы два фрагмента. Скомпонуйте на главном экране либо два фрагмента с помощью тега «fragment» и указанием соответствующего «name» в `main_activity.xml`, либо используйте метод `replace` в `FragmentManager`. Первый фрагмент должен содержать какой-то список (например страны). Допускается использовать списки типа `ListView`, `RecyclerView`. Второй экран содержит детальную информацию о выбранном пункте. При выборе пункта, должно производиться обновление содержимого экрана в `DetailsFragment`.

Fragment Result API

Задание. Создать новый модуль. В меню «File / New / New Module / Phone & Tablet Module / Empty Views Activity». Название модуля «ResultApiFragmentApp». Необходимо передать данные из одного фрагмента в другой.

В некоторых случаях может потребоваться передать единожды значение между двумя фрагментами или между фрагментом и активностью. В версии `Fragment 1.3.0` и выше каждый `FragmentManager` реализует `FragmentResultOwner`, тем самым `FragmentManager` может выступать в качестве центрального хранилища результатов фрагментов. Это позволяет компонентам взаимодействовать друг с другом, устанавливая результаты и прослушивая их, не требуя от компонентов прямых ссылок друг на друга.

`BottomSheetDialogFragment` - класс, который предоставляет возможность создавать диалоговые окна (Bottom Sheets). Они появляются в нижней части экрана и предоставляют пользователю дополнительное пространство для взаимодействия, сохраняя при этом текущий контекст приложения. `BottomSheetDialogFragment` расширяет функциональность стандартного `DialogFragment`.

Необходимо создать фрагмент `BottomSheetFragment`. Чтобы передать данные из одного фрагмента в другой, сначала необходимо установить слушателя в фрагменте `BottomSheetFragment`. Продумать внешний вид экрана фрагмента.

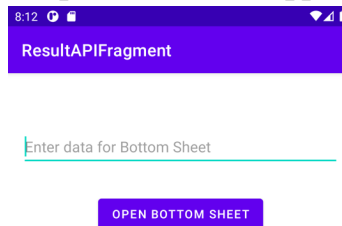
```

public class BottomSheetFragment extends BottomSheetDialogFragment {
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);

        getParentFragmentManager().setFragmentResultListener("requestKey", this, new
        FragmentResultListener() {
            @Override
            public void onFragmentResult(@NonNull String requestKey, @NonNull Bundle bundle) {
                String text = bundle.getString("key");
                Log.d(BottomSheetFragment.class.getSimpleName(), "Get text " + text);
            }
        });
    }
}

```

Необходимо создать файл разметки для фрагмента *DataFragment*.



В фрагменте *DataFragment* необходимо передать результат с помощью аналогичного *FragmentManager*, используя тот же *requestKey*.

```

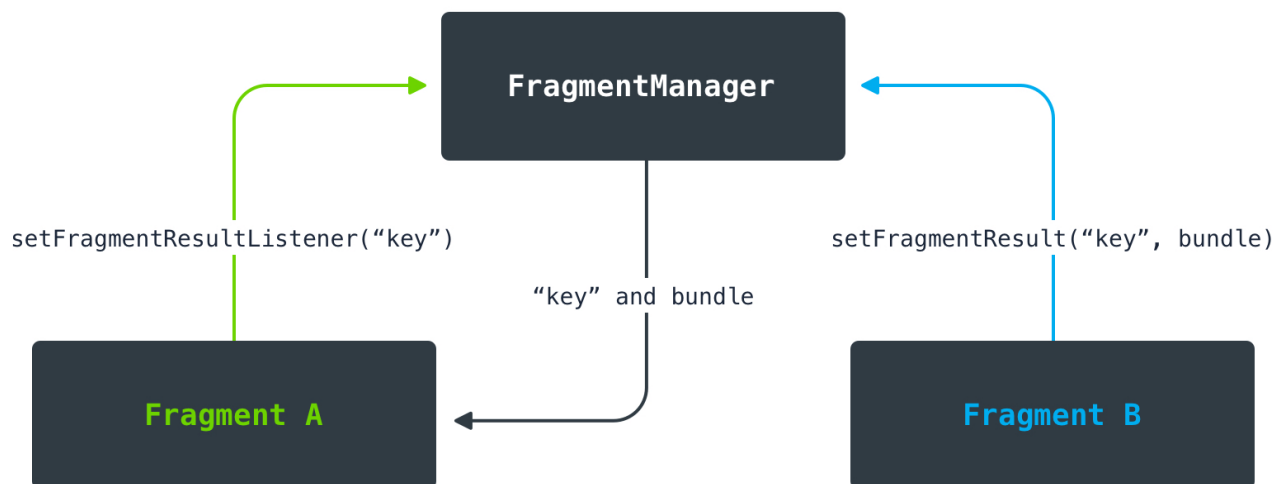
public class DataFragment extends Fragment {
    @Override
    public View onCreateView(LayoutInflater inflater, ViewGroup container,
        Bundle savedInstanceState) {
        View view = inflater.inflate(R.layout.fragment_data, container, false);
        Button button = view.findViewById(R.id.buttonOpenBottomSheet);

        button.setOnClickListener(click->{
            String text = ((EditText) view.findViewById(R.id.editTextInfo)).getText().toString();
            Bundle bundle = new Bundle();
            bundle.putString("key", text);
            getChildFragmentManager()
                .setFragmentResult("requestKey", bundle);

            BottomSheetFragment bottomSheet = new BottomSheetFragment();
            bottomSheet.show(getChildFragmentManager(),
                "ModalBottomSheet");
        });
        return view;
    }
}

```

Принцип взаимодействия фрагментов отображен на рисунке.



В данном случае фрагмент А получит результат, как только будет в состоянии `STARTED`. Если установить результат без соответствующего слушателя, результат будет храниться в *FragmentManager* до тех пор, пока не будет установлен слушатель с тем же ключом.

Чтобы получить результат фрагмента в Activity, необходимо установить слушателя с помощью функции *getSupportFragmentManager()*.

```
public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        getSupportFragmentManager().setFragmentManagerListener(
            "requestKey",
            this,
            (requestKey, bundle) -> {
                String result = bundle.getString("key");
                Log.d(BottomSheetFragment.class.getSimpleName(), "I'm MainActivity " + result);
            }
        );
    }
}
```

Использование интерфейса

Передачу данных между фрагментами можно также реализовать через использование интерфейса, посредником в данном случае выступает Activity. Для этого требуется создать интерфейс.

```
public interface FragmentListener {
    void sendResult(String message);
}
```

В MainActivity необходимо его имплементировать.

```

public class MainActivity extends FragmentActivity implements FragmentListener {

...

    @Override
    public void sendResult(String message) {
        Log.d(MainActivity.class.getSimpleName(), "message: " + message);
    }
}

```

Фрагмент *BlankFragment* должен предоставить публичный метод, который будет использован родительской Activity.

```

public class BlankFragment extends Fragment {
    private FragmentListener listener;

    @Override
    public void onAttach(Context context) {
        super.onAttach(context);
        try {
            listener = (FragmentListener) context;
        } catch (ClassCastException e) {
            throw new ClassCastException(context + " must implement onSomeEventListener");
        }
    }

    @Override
    public View onCreateView(LayoutInflater inflater, ViewGroup container,
                             Bundle savedInstanceState) {
        View view = inflater.inflate(R.layout.fragment_blank, container, false);
        ImageView image = view.findViewById(R.id.imageView);
        image.setOnClickListener(view1 -> listener.sendResult("image pushed"));
        return view;
    }
}

```

4. КОНТРОЛЬНОЕ ЗАДАНИЕ

В ходе выполнения практического задания № 1 были сформулированы use-case возможности проекта. Приложение должно удовлетворять следующим функциональным требованиям:

1. Авторизация в приложении.
2. Взаимодействия с каким-либо внешним сервисом, возвращающим данные о стоимости валюты, погоды и т.д. в виде json-формата. Возможно использовать сервисы типа <https://www.wiremock.io/>, <https://mockapi.io/>, <https://github.com/typicode/json-server>, <https://thecatapi.com/> и т.д.
3. Сохранение данных в БД.
4. Отображение списка каких-либо сущностей с изображениями.
5. Отображение страницы сущности.
6. Различные возможности у авторизованного и гостевого пользователя.
7. Использование обученной модели для распознавания изображений, аудио, маски и т.д. (нужно произвести анализ существующих обученных моделей TensorFlow Lite, которые желаете использовать в приложении).

Задание. Требуется открыть приложение из практической работы №1. Необходимо реализовать отображение списка из предыдущего занятия с использованием ViewModel и нескольких фрагментов. Добавить фрагмент «Профиль», который на основе авторизационных данных пользователя формирует экран. Для отображения экранов и навигации использовать фрагменты, учесть использование бэк-стека.